
Deep Learning

Lecture 03

Optimization

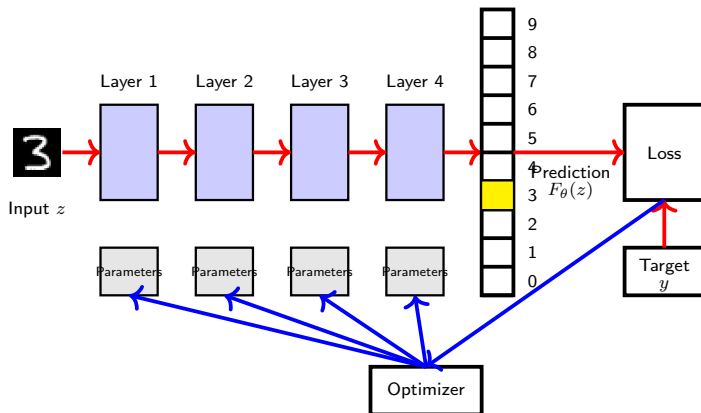
Aurelien Lucchi

Fall 2026

SECTION 1

Loss Function

Training adjusts the parameters to reduce a loss computed on minibatches.



Given target y and model output $F_{\theta}(z)$:

Regression: squared loss

$$\ell_y(F_{\theta}(z)) = \frac{1}{2} \|y - F_{\theta}(z)\|^2$$

Classification: cross-entropy

$$\ell_y(F_{\theta}(z)) = - \sum_{j=1}^m y_j \log p_{\theta,j}(z)$$

Language modeling: next-token cross-entropy

$$\ell(\theta; w_{1:T}) = - \sum_{t=1}^{T-1} \log p_{\theta}(w_{t+1} \mid w_{\leq t})$$

The loss determines the gradient signal. Metrics such as accuracy or exact match need not be differentiable.

Let $\mathcal{S} = \{(z_i, y_i)\}_{i=1}^n$ denote the training set.

Expected risk: performance on the unknown data distribution

$$\mathcal{R}(\theta) = \mathbb{E}_{(z,y) \sim \mathbb{P}} [\ell_y(F_\theta(z))]$$

Empirical risk: quantity available during training

$$\hat{\mathcal{R}}_{\mathcal{S}}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_{y_i}(F_\theta(z_i))$$

In practice we minimize a regularized objective

$$f(\theta) = \hat{\mathcal{R}}_{\mathcal{S}}(\theta) + \lambda \Omega(\theta).$$

Optimization reduces f . Generalization concerns the gap between training and unseen data.

Optimization question

- ▶ How quickly does training loss decrease?
- ▶ Does training remain numerically stable?
- ▶ How much compute and memory does each step require?

Generalization question

- ▶ Does validation loss decrease?
- ▶ Which solution does the optimizer select?
- ▶ How do regularization and data affect the result?

Two runs can reach similar training loss and still produce different validation performance. Optimizer, batch size, schedule, and weight decay affect both.

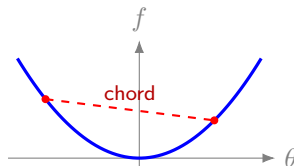
SECTION 2

Optimization Basics

A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is **convex** if, for all θ, θ' and $t \in [0, 1]$,

$$f(t\theta + (1 - t)\theta') \leq tf(\theta) + (1 - t)f(\theta').$$

The chord between two points lies above the graph.



Equivalent characterizations

- ▶ first order:
$$f(\theta') \geq f(\theta) + \nabla f(\theta)^\top (\theta' - \theta),$$
- ▶ second order: $H(\theta) = \nabla^2 f(\theta) \succeq 0$ for all θ .

Key consequence: every local minimizer is global, and $\nabla f(\theta^*) = 0$ implies global optimality.

Examples: linear and logistic regression are convex in θ ; neural-network losses are generally not.

Let f be twice continuously differentiable and θ^* a local minimizer.

Theorem 1

Necessary: $\nabla f(\theta^*) = 0$ and $H(\theta^*) \succeq 0$.

Sufficient: $\nabla f(\theta^*) = 0$ and $H(\theta^*) \succ 0 \Rightarrow$ *strict local minimizer*.

At a stationary point, the Hessian eigenvalues classify the local geometry:

Eigenvalues of $H(\theta^*)$	Stationary point
all > 0	local minimum
all < 0	local maximum
mixed signs	saddle point
some $= 0$, others ≥ 0	degenerate: second order is inconclusive

In overparameterized networks, minima are often **degenerate**.

The gradient of f is **L -Lipschitz** if

$$\|\nabla f(\theta) - \nabla f(\theta')\| \leq L \|\theta - \theta'\| \quad \forall \theta, \theta'.$$

The gradient cannot change faster than L times the distance moved.

For twice-differentiable f , this is equivalent to

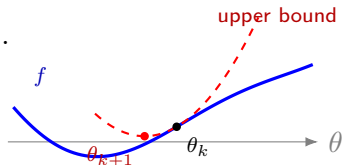
$$-LI \preceq H(\theta) \preceq LI \quad \Longleftrightarrow \quad \|H(\theta)\|_2 \leq L.$$

Consequence: f lies below a quadratic touching it at θ ,

$$f(\theta') \leq f(\theta) + \nabla f(\theta)^\top (\theta' - \theta) + \frac{L}{2} \|\theta' - \theta\|^2,$$

and minimizing this bound gives the gradient step with $\eta = 1/L$.

Example: $f(\theta) = \frac{1}{2} \theta^\top A \theta$ has $L = \lambda_{\max}(A)$.



SECTION 3

Deep Learning Geometry

Neural-network objectives are typically

- ▶ **non-convex**: stationary points need not be global minima,
- ▶ **high-dimensional**: models may contain billions of parameters,
- ▶ **overparameterized**: many parameter vectors can fit the data,
- ▶ **stochastic**: gradients come from minibatches,
- ▶ **scale-sensitive**: parameterization and normalization change the dynamics.

The classical question “does the method converge to the unique minimizer?” is usually replaced by:

Can we reach a useful solution, stably and within a compute budget?

For positively homogeneous activations such as ReLU,

$$\text{ReLU}(cu) = c \text{ReLU}(u), \quad c > 0.$$

Two consecutive layers can therefore represent the same function after

$$W_\ell \leftarrow cW_\ell, \quad W_{\ell+1} \leftarrow \frac{1}{c}W_{\ell+1}.$$

- ▶ Different parameter vectors can encode the same predictor.
- ▶ The Hessian can contain very small or zero eigenvalues.
- ▶ Euclidean gradient magnitudes depend on the chosen parameterization.

We will later see that normalization layers create additional scale invariances.

Near the current iterate θ , a second-order approximation gives

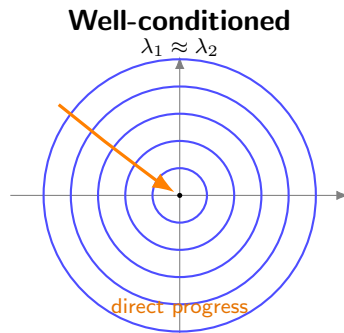
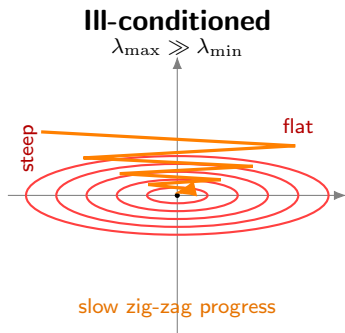
$$f(\theta + \Delta) \approx f(\theta) + \nabla f(\theta)^\top \Delta + \frac{1}{2} \Delta^\top H(\theta) \Delta, \quad H(\theta) = \nabla^2 f(\theta).$$

Curvature controls stability

- ▶ high curvature limits the learning rate,
- ▶ anisotropy causes slow progress,
- ▶ negative curvature reveals saddle directions.

But the model is local

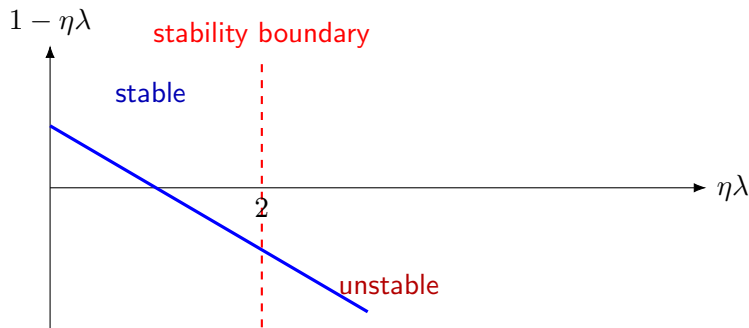
- ▶ the Hessian changes during training,
- ▶ stochastic gradients perturb the trajectory,
- ▶ large steps can leave the local region.



Sharpness and the Stability Threshold

For the one-dimensional quadratic $f(x) = \frac{\lambda}{2}x^2$,

$$x_{k+1} = (1 - \eta\lambda)x_k.$$



- ▶ Stability requires $|1 - \eta\lambda| < 1$, hence $0 < \eta\lambda < 2$.
- ▶ In neural-network training, the largest Hessian eigenvalue often evolves toward the threshold $2/\eta$.

SECTION 4

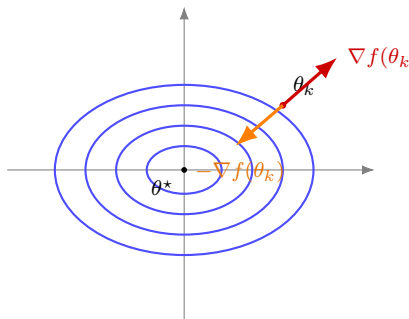
Gradient Descent

For a differentiable objective

$$f : \mathbb{R}^d \rightarrow \mathbb{R},$$

$$\theta_{k+1} = \theta_k - \eta_k \nabla f(\theta_k).$$

- ▶ $-\nabla f(\theta_k)$ is the steepest local descent direction.
- ▶ The learning rate η_k controls both progress and stability.



Mini-batching Deep-learning training replaces the full gradient with a minibatch estimate.

If f has an L -Lipschitz gradient, then

$$f(\theta + \Delta) \leq f(\theta) + \nabla f(\theta)^\top \Delta + \frac{L}{2} \|\Delta\|^2.$$

With $\Delta = -\eta \nabla f(\theta)$,

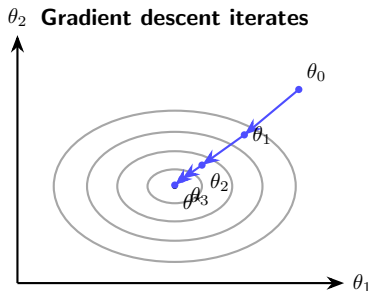
$$f(\theta_{k+1}) \leq f(\theta_k) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla f(\theta_k)\|^2.$$

Thus, $\eta \leq 1/L$ guarantees

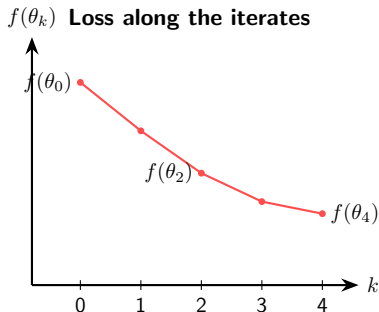
$$f(\theta_{k+1}) - f(\theta_k) \leq -\frac{\eta}{2} \|\nabla f(\theta_k)\|^2.$$

Practical limitation: a single global L can be extremely conservative and the local curvature changes throughout training.

The Descent Lemma



For a sufficiently small step size, gradient descent moves toward lower values of the objective.



The descent lemma guarantees monotonic decrease when the step size is small enough.

For a quadratic with Hessian $H = U\Lambda U^\top$, gradient descent evolves independently along each eigenvector:

$$\tilde{\theta}_{k+1,i} = (1 - \eta\lambda_i)\tilde{\theta}_{k,i}.$$

- ▶ The largest eigenvalue limits the stable learning rate.
- ▶ The smallest positive eigenvalue controls slow progress.
- ▶ A large condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ creates very different time scales.

Preconditioning replaces the update by

$$\theta_{k+1} = \theta_k - \eta P_k \nabla f(\theta_k),$$

where P_k rescales directions. Adam uses a diagonal, data-dependent P_k .

Theorem 2

Let f be differentiable, L -smooth, and bounded below by f^ . Gradient descent with $\eta \leq 1/L$ satisfies*

$$\min_{0 \leq i < K} \|\nabla f(\theta_i)\|^2 \leq \frac{2(f(\theta_0) - f^*)}{\eta K}.$$

The theorem guarantees that some iterate approaches stationarity. It does **not** guarantee:

- ▶ a global minimum,
- ▶ good validation performance,
- ▶ a useful rate for realistic learning rates and models.

Theory supplies useful mechanisms and diagnostics, but practical training is more complex.

SECTION 5

Stochastic Gradient Descent

For

$$f(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta),$$

sample a minibatch $\mathcal{B}_k$ of size B and form

$$g_k = \frac{1}{B} \sum_{i \in \mathcal{B}_k} \nabla f_i(\theta_k).$$

Stochastic gradient descent uses

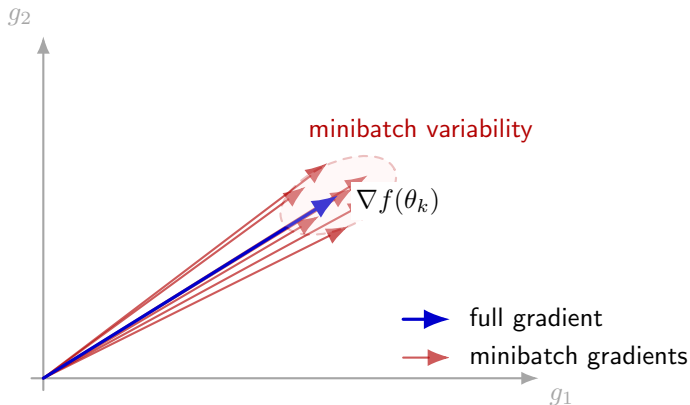
$$\theta_{k+1} = \theta_k - \eta_k g_k.$$

For uniform sampling,

$$\mathbb{E}[g_k \mid \theta_k] = \nabla f(\theta_k).$$

One optimizer step now costs B examples instead of the complete data set. Modern training usually measures B in **samples** for vision and **tokens** for language models.

Unbiased but Noisy Gradients



The mean direction matches the full gradient, but each update follows one noisy realization.

Write the minibatch gradient as

$$g_k = \nabla f(\theta_k) + \varepsilon_k, \quad \mathbb{E}[\varepsilon_k \mid \theta_k] = 0.$$

Under independent sampling, the covariance scales approximately as

$$\text{Cov}(g_k \mid \theta_k) \approx \frac{1}{B} \Sigma(\theta_k).$$

- ▶ Larger batches reduce gradient noise.
- ▶ Smaller batches perform more updates for a fixed number of examples.
- ▶ Noise is anisotropic and changes during training.

The batch size therefore changes both systems efficiency and optimization dynamics.

Increasing the batch size has two competing effects.

Potential benefit

- ▶ more parallel work per step,
- ▶ lower gradient variance,
- ▶ fewer synchronization points per token.

Potential cost

- ▶ more memory and communication,
- ▶ fewer updates per epoch,
- ▶ diminishing reduction in steps beyond a critical batch size.

Below the critical regime, a larger batch can often support a larger learning rate. Beyond it, extra examples mainly improve hardware utilization rather than sample efficiency.

Goyal et al., 2017; McCandlish et al., 2018.

For a smooth non-convex objective and a stochastic gradient with bounded second moment, a typical bound has the form

$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbb{E} \|\nabla f(\theta_k)\|^2 \leq \underbrace{\frac{2(f(\theta_0) - f^*)}{\eta K}}_{\text{optimization}} + \underbrace{L\eta \frac{\sigma^2}{B}}_{\text{gradient noise}} .$$

- ▶ A larger η improves the first term but worsens the noise term.
- ▶ A larger batch reduces the noise term.
- ▶ Decaying the learning rate lowers the late-training noise floor.

This simple tradeoff already motivates minibatching and learning-rate schedules.

SECTION 6

Momentum and Adaptivity

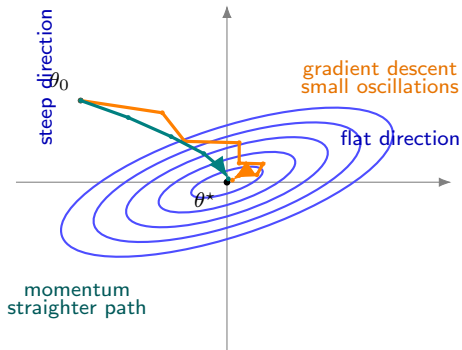
Momentum maintains an exponentially weighted average of past gradients:

$$v_{k+1} = \beta v_k + g_k, \quad \theta_{k+1} = \theta_k - \eta_k v_{k+1}.$$

- ▶ Consistent directions accumulate.
- ▶ Oscillations in high-curvature directions are damped.
- ▶ The effective step depends on both η_k and β .

Exercise: If the gradient stays constant, show that $v_k \rightarrow \frac{1}{1-\beta} g$. For $\beta = 0.9$, the steady-state magnitude is ten times the raw gradient.

Illustration Momentum



Adam combines momentum with coordinatewise scaling:

$$\begin{aligned}m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k, \\v_k &= \beta_2 v_{k-1} + (1 - \beta_2) (g_k \odot g_k), \\ \hat{m}_k &= \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}, \\ \theta_{k+1} &= \theta_k - \eta_k \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}.\end{aligned}$$

Interpretation: the numerator smooths the direction, while the denominator reduces updates in coordinates with persistently large gradients.

Adam is attractive for Transformers because gradient scales vary substantially across parameters and layers.

Adding an ℓ_2 term to the loss sends $\lambda\theta_k$ through Adam's coordinatewise preconditioner. AdamW instead applies decay directly:

$$\theta_{k+1} = (1 - \eta_k \lambda) \theta_k - \eta_k \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}.$$

- ▶ The decay does not inherit Adam's coordinatewise scaling.
- ▶ Learning rate and weight decay remain coupled through $\eta_k \lambda$.
- ▶ Biases and normalization scale parameters are often excluded from decay.

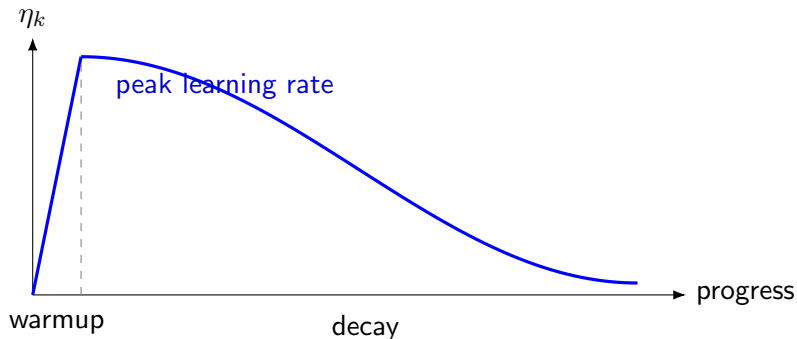
AdamW is a standard baseline for Transformer training.

Loshchilov and Hutter, 2019, [Decoupled Weight Decay Regularization](#).

SGD with Momentum or AdamW?

	SGD + momentum	AdamW
State per parameter	one momentum buffer	first and second moments
Scaling	one global scale	coordinatewise adaptive scale
Common use	CNN training, large labeled data	Transformers, fine-tuning, sparse gradients
Main sensitivity	learning rate, momentum	learning rate, β_2 , ϵ , decay
Memory	lower	higher

Neither method dominates in every setting. Compare them under the same token or compute budget, and tune the learning-rate schedule for each optimizer.



- ▶ **Warmup** limits early updates while optimizer moments and activation scales settle.
- ▶ **Decay** reduces late-training noise and enables refinement.
- ▶ Cosine and linear decay are common. A stable phase can precede decay.

Global norm clipping replaces g_k by

$$\tilde{g}_k = g_k \min \left\{ 1, \frac{c}{\|g_k\|_2 + \epsilon} \right\}.$$

- ▶ The direction is unchanged when clipping activates.
- ▶ The update cannot be dominated by one unusually large gradient norm.
- ▶ Clipping is especially common for recurrent networks and Transformers.

One optimizer configuration need not apply to every tensor.

Parameter group	Typical treatment	Reason
Dense matrix weights	adaptive optimizer, weight decay	most model capacity
Biases	often no weight decay	decay has little structural motivation
Normalization scales	often no weight decay	directly control activation scale
Embeddings or output head	sometimes separate learning rate	scale and statistics differ

Architectures with tied embeddings, low-rank adapters, or mixture-of-experts may need additional parameter groups. Record the grouping in every reproducible training configuration.

Adam rescales each coordinate independently. Matrix-based optimizers also use relationships between rows and columns of weight matrices.

Shampoo

- ▶ approximates full-matrix AdaGrad with Kronecker factors.

Muon

- ▶ orthogonalizes momentum updates for two-dimensional hidden weights,
- ▶ commonly keeps AdamW for embeddings, biases, and normalization parameters.

These methods can improve progress per optimizer step, but wall-clock gains depend on matrix shapes, communication, and implementation quality.

Gupta et al., 2018; Shi et al., 2023; Liu et al., 2025.

SECTION 7

Training at Scale

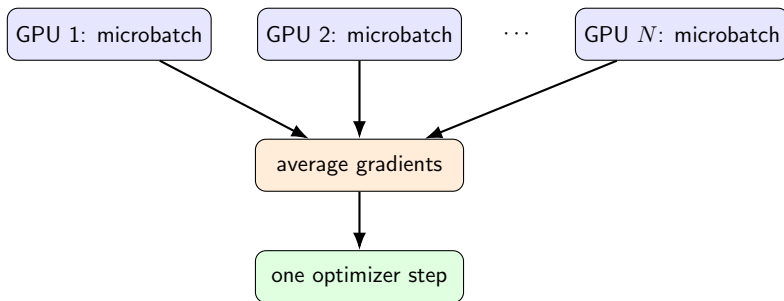
Modern accelerators provide much higher throughput at low precision.

- ▶ Use FP16 or BF16 for most activations and matrix multiplications.
- ▶ Keep numerically sensitive reductions and optimizer state in higher precision.
- ▶ FP16 often needs dynamic loss scaling to avoid gradient underflow.
- ▶ BF16 has the exponent range of FP32 and usually needs less loss scaling.

Mixed precision leaves the objective unchanged, but overflow, underflow, and rounding can destabilize training.

With data parallelism and gradient accumulation,

$$B_{\text{global}} = B_{\text{micro}} \times N_{\text{devices}} \times N_{\text{accumulation}}.$$



Accumulation reproduces a larger batch mathematically when gradients are summed at the same parameters. It does not provide the same wall-clock speed as true parallelism.

Training loss alone is insufficient.

Quantity	What it can reveal
Validation loss or task metric	overfitting, data mismatch, checkpoint quality
Learning rate	schedule bugs and incorrect resume behavior
Gradient norm	spikes, clipping frequency, exploding gradients
Update-to-weight ratio	layers that barely move or receive excessive updates
Activation and parameter norms	scale drift, saturation, normalization issues
Throughput and utilization	input bottlenecks, communication overhead, stragglers

1. **Check the data and loss**: inspect batches, masks, labels, and reduction conventions.
2. **Lower the peak learning rate**: divergence often appears first as a gradient-norm spike.
3. **Inspect precision**: check overflows, loss scaling, and numerically sensitive operations.
4. **Clip gradients**: use this as protection and as a diagnostic.
5. **Verify scaling**: confirm global batch size, gradient averaging, accumulation, and scheduler step count.
6. **Simplify**: reproduce on one device and a small batch before restoring distributed training.

1. Deep-learning optimization is non-convex, stochastic, and strongly shaped by parameterization.
2. The learning rate and batch size jointly control progress, stability, and gradient noise.
3. Momentum smooths directions. AdamW adds coordinatewise scaling and decoupled weight decay.
4. Schedules, clipping, precision, and parameter groups form part of the optimizer rather than incidental implementation details.

- ▶ Kingma and Ba (2015), **Adam: A Method for Stochastic Optimization**.
- ▶ Loshchilov and Hutter (2019), **Decoupled Weight Decay Regularization**.
- ▶ Goyal et al. (2017), **Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour**.
- ▶ McCandlish et al. (2018), **An Empirical Model of Large-Batch Training**.
- ▶ Cohen et al. (2021), **Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability**.
- ▶ Gupta et al. (2018), **Shampoo: Preconditioned Stochastic Tensor Optimization**.
- ▶ Shi et al. (2023), **A Distributed Data-Parallel PyTorch Implementation of Shampoo**.
- ▶ Liu et al. (2025), **Muon is Scalable for LLM Training**.